
Double-Descent Under Redundant Training Data

Felipe Lorenzi
UC San Diego
florenzi@ucsd.edu

1 Introduction

A striking feature of modern overparameterized models is that the classic bias-variance tradeoff does not exactly apply. Instead, test error can follow a *double-descent* curve: it first decreases, then spikes near interpolation, and then decreases again in the overparameterized regime (Belkin et al., 2019). This behavior has been studied in linear models, kernels, and random feature models, and it is one of the cleanest examples of why the classical underfit/overfit picture is incomplete (Bartlett et al., 2020; Mei and Montanari, 2022).

The question I wanted to study here is simple: **what happens to double descent when the training data are redundant?** This matters because many real datasets are not made of fully independent examples. Time series and spatial observations can be highly correlated, and large web-scale datasets often contain repeated or near-repeated content. Intuitively, if many training rows contain almost the same information, then the raw number of observations may actually overstate the number of genuinely distinct observations the model sees.

This question is especially relevant for two types of modern datasets. First, many scientific datasets are spatial or temporal, where nearby observations are strongly correlated and may contain very similar information. In those settings, the sample size may not reflect the true number of independent observations. Second, extremely large web-scale datasets used to train large language models actually likely contain substantial amounts of redundancy, including repeated documents, paraphrases, and near-duplicate text. Even when the dataset contains billions of tokens, much of the information may be repeated many times. These situations raise a natural question: does the interpolation threshold behave as though the dataset were effectively smaller when the training data are highly redundant?

To keep the experiments simple and fast, I work in a fixed random feature setting. Labels are generated by a linear teacher, while the student is a random ReLU feature model that only trains its output weights. This gives a model that is nonlinear in the input but still easy to solve exactly. I then compare three cases: no redundancy, exact repeated samples, and near-duplicates obtained by adding small perturbations to repeated samples.

The main conclusion is that exact duplicates shift the interpolation peak in a very clean way: the peak tracks the number of unique training samples rather than the total number of rows. Near-duplicates are different. They do not move the dominant peak nearly as well, but they significantly worsen conditioning of the problem and can create smaller "bumps" before the main interpolation peak. Therefore, the experiments support a simple picture: exact redundancy changes the number of independent pieces of information, while approximate redundancy mainly changes numerical stability.

2 Related work

The double-descent curve was first observed by Belkin et al. (2019), who argued that the classical bias-variance tradeoff misses an important regime that appears once models are allowed to interpolate the data. Related work then showed that interpolation can still generalize well in some settings, especially in high-dimensional regression problems (Bartlett et al., 2020).

Random feature models are especially useful here because they sit between simple linear models and wide neural networks. They are nonlinear in the input, but after the random first layer is fixed, the remaining training problem is just linear regression. This makes them a standard for testing theory related to the double-descent curve (Mei and Montanari, 2022).

3 Methods

3.1 Data-generating process and Random ReLU model

I use a teacher–student construction, where data are generated by the teacher, and the model we analyze is the student.

The teacher is linear:

$$y = x^\top \beta,$$

where $x \in \mathbb{R}^d$ and $\beta \in \mathbb{R}^d$ is a fixed random unit vector. In all experiments, $d = 100$. Using a linear teacher keeps the ground truth simple, which makes results easier to interpret.

The student is a fixed random ReLU feature model. I draw

$$W \in \mathbb{R}^{M \times d}, \quad W_{jk} \sim \mathcal{N}(0, 1),$$

and for each width m , I use the first m rows of W to define the feature map

$$z(x) = \text{ReLU}\left(\frac{W_m x}{\sqrt{d}}\right) \in \mathbb{R}^m.$$

The predictor is

$$\hat{y}(x) = a^\top z(x),$$

where only the output weights a are trained.

Given training feature matrix $Z \in \mathbb{R}^{n \times m}$ and labels $y \in \mathbb{R}^n$, I solve for a using a ridge-regression linear system:

$$\hat{a} = \arg \min_a \|Za - y\|_2^2 + \lambda \|a\|_2^2,$$

with very small $\lambda = 10^{-6}$ kept for numerical stability, but small enough that it does not hide the interpolation peak of the double-descent curve. Since the first layer is fixed, this gives a direct and reproducible way to study interpolation as width changes.

3.2 Data construction

The total number of training rows is fixed at

$$n_{\text{train}} = 256,$$

and the test set has

$$n_{\text{test}} = 4000$$

independent points drawn from the same distribution. I sweep model width from very small values up to $m = 1024$, with higher resolution around the interpolation region near $m \approx 256$.

I consider three data settings:

Baseline (no redundancy): All 256 training inputs are unique i.i.d. Gaussian vectors.

Exact duplicates: I choose a duplication factor $r \in \{1, 2, 4, 8\}$. I first sample

$$n_{\text{unique}} = \frac{256}{r}$$

unique points, then repeat each r times. This creates a training matrix with 256 rows but only n_{unique} unique observations. The four cases I test, in terms of number of unique observations, are therefore:

$$n_{\text{unique}} \in \{256, 128, 64, 32\}.$$

Near-duplicates: Starting from the exact-duplicate construction, I add small Gaussian jitter to each repeated row. In the runs shown here, the jitter scale is 0.02. This keeps repeated samples close to each other without making them exactly identical.

3.3 Metrics

For each width, I record train MSE, test MSE, and the smallest singular value and condition number of the training feature matrix Z . The condition number and singular values help explain whether the test-error peak is really tied to interpolation and poor conditioning.

The main question is not just whether a spike appears, but where it appears. In the duplicate experiments, the most natural candidate for an effective sample size is simply the number of unique training samples, n_{unique} .

4 Results

4.1 Baseline and exact duplicates

The baseline experiment with unique training points shows the classic double-descent picture: as width increases, test error decreases at first, then blows up near the interpolation threshold, and then drops again in the overparameterized regime. In this setup, the peak appears near $m \approx n_{\text{train}} = 256$, which is exactly the behavior expected.

The exact-duplicate experiments are the main result of this project. When the training set contains repeated rows, the peak shifts left in a very clean way. For duplication factors $r = 2, 4, 8$, the test-error peak moves from around $m \approx 256$ to around

$$m \approx 128, \quad m \approx 64, \quad m \approx 32,$$

respectively. These values match the number of unique training samples almost exactly.

Figure 1 shows this result in two ways. The left panel plots test MSE against raw width m , where the peak clearly moves left as duplication increases. The right panel replots the same curves against m/n_{unique} . If redundancy really reduces the number of distinct constraints the model is learning, then plotting error against m/n_{unique} should align the peaks better than plotting against raw width m . In this rescaled view, the peaks align much better and occur near

$$\frac{m}{n_{\text{unique}}} \approx 1.$$

This is strong evidence that, in the exact-duplicate setting, the effective interpolation threshold is controlled by the number of distinct constraints the model is learning rather than the raw number of rows.

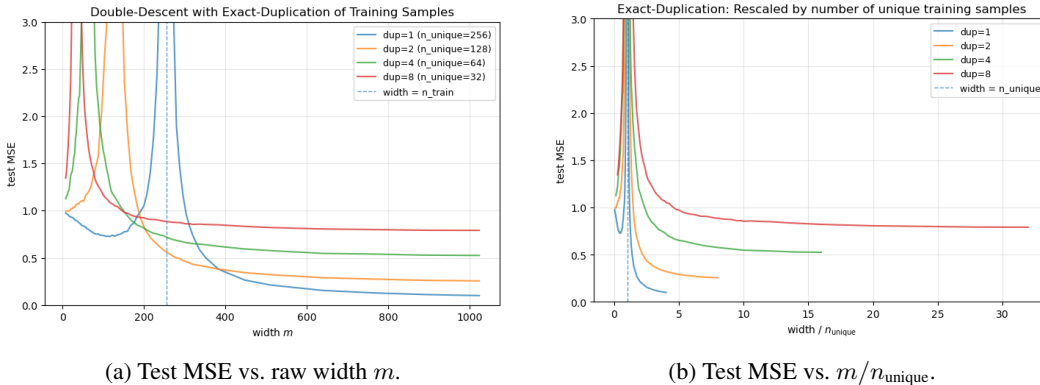


Figure 1: Main duplicate-data result. Exact repeated samples shift the double-descent peak from the raw sample size toward the number of unique training samples. Rescaling by n_{unique} produces a near-collapse of the curves.

The training-error and conditioning plots tell the same story. In Figure 2, the train MSE falls to essentially zero much earlier when there are fewer unique samples, which shows that interpolation itself has shifted left toward smaller widths. At the same time, the condition number of the feature matrix Z peaks at roughly the same shifted widths. So the test-error peak moves because the redundant dataset imposes fewer distinct constraints for the model to learn, and the design matrix becomes badly conditioned when the width of the model approaches that reduced constraint count.

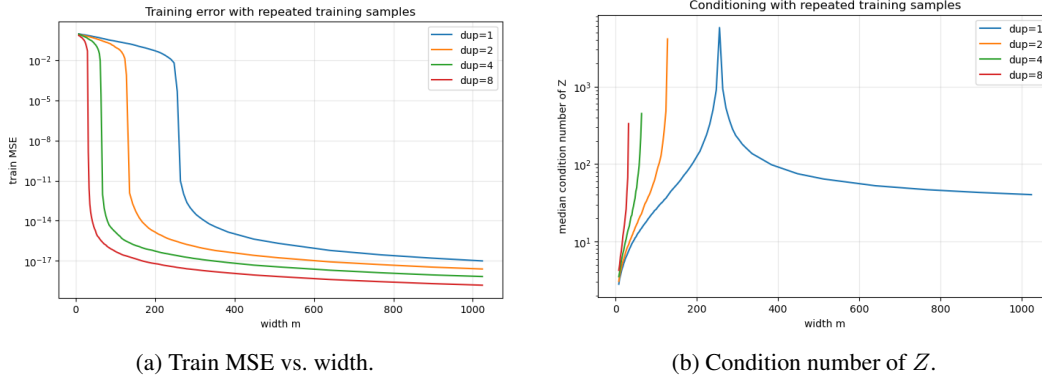


Figure 2: Diagnostics for exact duplicates. The train-error transition and the conditioning spike both shift in the same direction as the test-error peak, supporting the interpretation that the effective interpolation threshold is determined by the number of unique constraints.

4.2 Near-duplicates

The near-duplicate experiment changes the picture in an important way. Here I start with repeated samples but add small jitter so that the rows are similar rather than exactly identical. The result is more subtle. The training set is still highly redundant, but the rows are no longer literally the same, so the number of exact constraints is not reduced as sharply.

Figure 3 shows the two most important patterns. First, the dominant test-error peak no longer shifts cleanly to $m \approx n_{\text{unique}}$. In the runs shown here, the largest peak stays near the raw interpolation width $m \approx 256$ across duplication factors. Second, redundancy still matters a lot: the condition number becomes much larger much earlier than in the baseline, and the test-error curves can develop smaller bumps at earlier widths due to the redundancy in the data.

So near-duplicates do not behave just like exact duplicates. They do not reduce the number of constraints enough to move the main peak in a clean one-to-one way. Instead, they mainly make the interpolation region less stable. This is a useful result because it separates two ideas that are easy to confuse: exact redundancy can change the effective interpolation threshold itself, while approximate redundancy can leave the threshold roughly where it was but greatly worsen conditioning.

4.3 Interpretation

Taken together, the experiments suggest the following interpretation. During training, the model is effectively trying to solve a system of equations of the form

$$Za \approx y,$$

where Z is the feature matrix and each row corresponds to one training example. Each training point therefore contributes one equation that the model tries to satisfy. If all training examples are different, these equations are typically independent, and the interpolation threshold occurs when the number of model parameters (the width m) is roughly equal to the number of training rows n_{train} .

Exact duplicates change this situation in a simple way. When the same training point appears multiple times with the same label, it produces identical rows in the feature matrix Z . These rows correspond to the same equation repeated multiple times, so they do not add new information. In linear algebra terms, this creates linear dependence between rows of Z and reduces its rank. In the experiments, we can see that the relevant number becomes the number of unique samples n_{unique} , which explains why the interpolation peak shifts from $m \approx n_{\text{train}}$ toward $m \approx n_{\text{unique}}$.

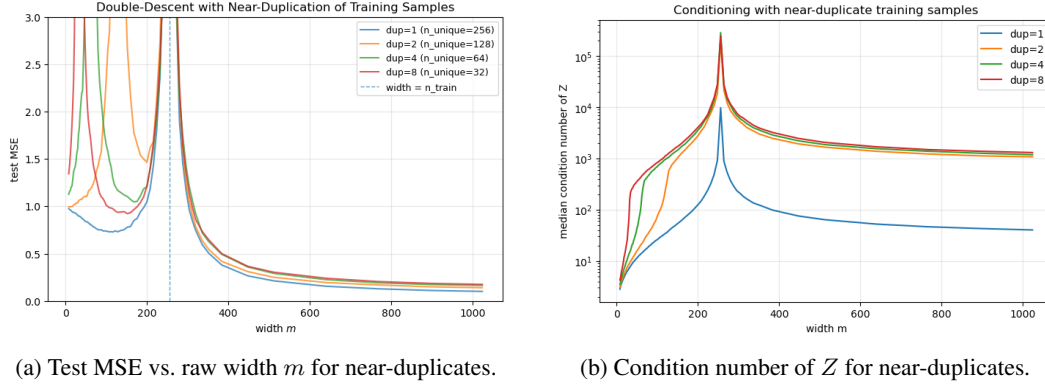


Figure 3: Near-duplicate extension. Small perturbations to repeated samples do not move the dominant peak as cleanly as exact duplication, but they still make the design matrix much more ill-conditioned and can create smaller early bumps before the main interpolation spike.

Near-duplicates behave differently. When small jitter is added to repeated samples, the rows of Z are no longer identical, so they are not exactly linearly dependent. The matrix therefore still has nearly full rank, and the number of independent equations remains close to n_{train} . For this reason, the dominant interpolation peak still appears near $m \approx n_{\text{train}}$. However, the rows of Z become very similar, which makes the matrix poorly conditioned. In practice this means the smallest singular values of Z become very small, and the solution to the regression problem becomes extremely sensitive to small changes in the data. This loss of numerical stability explains why near-duplicates still increase the size of the double-descent peak and can produce smaller earlier bumps at lower widths, even though they do not move the main peak as cleanly as exact duplicates.

5 Discussion and conclusion

This project was meant to be a small synthetic study of a simple question: how does redundant data change double descent? The motivation comes from settings where the raw dataset size may not reflect the amount of independent information in the data. This occurs in spatial or temporal datasets, where nearby observations can be strongly correlated, and in large web-scale datasets used for training modern large language models, where the same ideas or even entire documents may appear many times in slightly different forms.

When redundancy is exact, the effect is clean and intuitive. The model needs to fit many repeated versions of the same constraint, so the relevant sample count is no longer the number of rows in the dataset but the number of distinct rows. In that regime, the double-descent peak shifts almost exactly with the number of unique training samples, and the rescaled curves line up well.

When redundancy is only approximate, the story changes. Small perturbations prevent a literal collapse in the number of constraints, so the feature matrix remains nearly full rank, and the main interpolation peak stays near the raw training size. Even so, the feature matrix becomes much more ill-conditioned, and that increased instability shows up in both the conditioning diagnostics and the test-error curves.

This is the most useful takeaway from the project: It would be too strong to say that *all* correlation or redundancy simply reduces the sample size from n to some smaller effective value. It seems that the effect is that exact redundancy behaves like a reduced constraint count, while weaker redundancy acts more like a conditioning problem. Both effects matter, but they are not necessarily the same.

There are several natural extensions to this project. One would be to study forms of redundancy that are more realistic than either exact duplicates or small independent jitter. For example, one could generate data from a lower-dimensional latent variable and then map that latent variable into a higher-dimensional observed space. In that case, the samples would not be literal duplicates, but they would still lie near a lower-dimensional structure and therefore contain less independent variation than the ambient dimension suggests. Another option would be blockwise repeated structure, where groups of nearby samples share the same underlying signal plus a small amount of local variation.

This would mimic settings where observations come in clusters or temporally correlated data. Another useful extension would be to compare different notions of effective rank or effective dimension, rather than using only the literal number of unique samples. That would give a more flexible way to quantify redundancy in datasets where repetition is approximate rather than exact. Finally, it would be interesting to test whether similar patterns appear in trained neural networks rather than fixed random feature models. Still, even this simple experiment already gives a useful takeaway: in overparameterized regression, the behavior near interpolation depends not just on how many rows are in the dataset, but on how many genuinely distinct pieces of information those rows contain. When redundancy is strong enough to create repeated constraints, the interpolation threshold can shift substantially.

References

- Bartlett, P. L., Long, P. M., Lugosi, G., and Tsigler, A. Benign overfitting in linear regression. *Proceedings of the National Academy of Sciences*, 117(48):30063–30070, 2020.
- Belkin, M., Hsu, D., Ma, S., and Mandal, S. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.
- Hastie, T., Montanari, A., Rosset, S., and Tibshirani, R. J. Surprises in high-dimensional ridgeless least squares interpolation. *The Annals of Statistics*, 50(2):949–986, 2022.
- Mei, S. and Montanari, A. The generalization error of random features regression: Precise asymptotics and the double descent curve. *Communications on Pure and Applied Mathematics*, 75(4):667–766, 2022.